

基于代理模型预测序列长度的 高效交互式 LLM 服务

Haoran Qiu
UIUC

Weichao Mao
UIUC

Archit Patke
UIUC

Shengkun Cui
UIUC

Saurabh Jha
IBM Research

Chen Wang
IBM Research

Hubertus Franke
IBM Research

Zbigniew Kalbarczyk
UIUC

Tamer Başar
UIUC

Ravishankar Iyer
UIUC

摘要

大型语言模型 (LLM) 正在众多领域推动新一轮交互式人工智能应用。然而，高效处理 LLM 推理请求十分困难，因为生成模型的自回归特性导致执行时间难以预测。现有 LLM 服务系统采用先到先服务 (FCFS) 调度，存在队头阻塞问题。

为应对 LLM 执行时间的不确定性、提高交互式服务效率，我们提出推测式最短作业优先 (SSJF) 调度器，使用轻量代理模型预测 LLM 输出序列长度。开源 SSJF 实现无需修改内存管理或组批策略。在真实数据集和生产工作负载轨迹上的评估表明，与 FCFS 相比，SSJF 在无组批、动态组批、连续组批三种设置下，将平均作业完成时间降低 30.5–39.6%，吞吐量提高至 2.2–3.6 $\times$ 。

关键词

大型语言模型，模型服务，调度

1 引言

在海量数据上训练的大型语言模型 (LLM) 取得了新进展，能够进行类似人的对话，并处理机器翻译、编程、创意写作等多种任务。随着 LLM 持续发展，它们在众多领域推动新一轮交互式人工智能应用。因此，高效推理服务对用户体验十分重要：交互性要求推理请求具有较低的作业完成时间 (JCT) 与较高吞吐量。

挑战。传统深度神经网络 (DNN)，例如 ResNet、BERT，其推理请求执行时间具有确定性 [11]；LLM (例如 GPT) 服务则面临执行时间不可预测的挑战。这源于生成式 LLM 的自回归特性：对一个输入请求，LLM 以迭代方式生成输出 token 序列，每次生成一个 token [28]；生成的 token 被追加到输入序列，以供下一次迭代生成下一个 token。因此，处理请求所需的执行时间和总迭代次数事先未知。

Clipper [6]、TensorFlow-Serving [20]、Clockwork [11] 等传统 DNN 服务系统面向确定性推理请求。相同硬件上，同一模型各请求执行时间相同，因此采用先到先服务 (FCFS) 策略。Orca [28]、vLLM [15]、AlpaServe [17] 等先进 LLM 服务系统仍沿用 FCFS。然而，FCFS 会造成队头阻塞 [22, 26]；LLM 模型很大、绝对执行时间较长，使问题更严重。FastServe [26] 通过多级反馈队列改善平均完成时间，但没有直接处理本文关注的执行时间不确定性，而且需要保存未完成作业的中间状态，产生额外内存管理开销。

本文工作。我们的关键观察是，小型代理模型，即经过微调的 BERT-base，可以根据输入查询较好地预测 LLM 回答的长短。基于此，我们设计并实现了推测式最短作业优先 (SSJF) 调度器，用代理模型预测序列长度、估计执行时间。SSJF 可直

接应用于现有 LLM 服务系统，无需修改内存或键/值缓存管理；也支持不同组批设置：无组批 (例如 [17])、动态组批 (例如 [1])、连续或迭代级组批 (例如 [28])。

结果。我们使用常见开源 LLM、真实 LLM 对话数据集 [30]，在 NVIDIA V100 上以生产工作负载轨迹驱动，评估 SSJF。评估涵盖无组批、动态组批、连续组批三种设置。在不同请求速率与突发程度下，相对于 FCFS，SSJF 将平均 JCT 降低 30.5–39.6%，吞吐量提高至 2.2–3.6 $\times$ 。

贡献。本文的主要贡献如下：

- 提出使用轻量代理模型的推测式请求调度器 SSJF，应对生成式 LLM 执行时间不确定性。
- 开源实现 SSJF，并在真实模型数据集和生产轨迹上评估。
- 除请求调度外，讨论代理模型在 LLM 服务中的其他潜在用途，见 §6。

2 背景与动机

交互式 LLM 应用服务。大型语言模型 (LLM)，例如 GPT，是基于 Transformer 的生成式模型，支持聊天机器人 [21]、人工智能结对编程 [9]、搜索 [10] 等交互式应用。LLM 以自回归方式生成输出 token 序列。因此，处理生成模型请求需要运行模型的多次迭代：每次生成一个输出 token，并在下一次迭代将其追加到原始输入中，终止 token <EOS> 除外。

自回归模式导致 LLM 推理服务的执行时间不确定。我们分析了 LMSYS-Chat-1M 数据集 [30] 的输出 token 长度分布；该数据集包含与 25 个先进 LLM 的 100 万次真实对话。对每个模型实例，输出长度都有明显差异，各模型输出长度的 p95/p50 比值从 1.7 (claude-1) 到 20.5 (llama-13b) 不等。输出 token 长度 (N) 主导请求执行时间 (T)，因为 $T = C + K * N$ ；其中， K 为生成一个 token 的时延， C 为模型服务系统开销，包括 DNS 查询、代理、排队、输入 token 化。 K 取决于量化等模型优化技术与硬件等执行环境，对所有输入相同。

调度与组批。机器学习模型服务系统封装底层执行引擎，对到达请求排队，将作业派发到可用 GPU 等计算设备，即调度，并把结果返回给用户。服务系统通常对请求组批，利用加速器中的并行计算单元提高硬件利用率和吞吐量。批次中的输入被拼接后整体送入模型。然而，自回归模型执行时间不确定，给调度和组批都带来挑战。

首先，LLM 服务采用 FCFS 时可能发生队头阻塞，导致较长等待时间与作业完成时间 (JCT)。传统 DNN 服务系统平等处理同一模型实例的请求，采用 FCFS，因为 ResNet、BERT 等模型的同实例具有确定性执行时间 [5, 6, 11, 20]。FastServe [26] 采用多级反馈队列改善平均 JCT，但没有直接预测执行时间的不确定性。

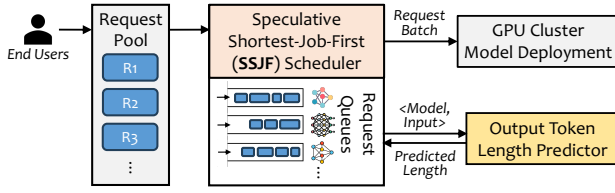


图 1: SSJF 概览。图内英文: End Users 为终端用户, Request Pool 为请求池, Request Queues 为请求队列, Speculative Shortest-Job-First Scheduler 为推测式最短作业优先调度器; Output Token Length Predictor 为输出 token 长度预测器, Predicted Length 为预测长度, Model、Input 分别为模型和输入, Request Batch 为请求批次, GPU Cluster、Model Deployment 分别为 GPU 集群和模型部署。

其次, 把不同输出长度的请求放入同一批次, 可能使已提前完成的请求等待批次内未完成的长请求, 降低硬件利用率。Orca [28] 提出迭代级调度与连续组批, 当前一批次中有请求提前结束时, 重新派发一个新批次。然而, Orca 使用 FCFS, 仍可能出现队头阻塞和较长 JCT。

3 方法

用户输入请求在运行时被派发到相应模型。每个模型实例维护一个请求队列, SSJF 使用推测式最短作业优先调度器决定执行顺序。SJF 缓解生成模型执行时间不确定带来的影响与 FCFS 队头阻塞。由于实际执行时间未知, 而 SJF 需要这项信息, SSJF 依赖输出 token 长度预测器, 见图 1。如 §2 所述, 输出长度主导执行时间, 两者近似线性相关, 因此预测用于估计 SJF 的作业执行时间。预测器采用轻量代理模型, 本文使用 BERT-base。

3.1 用于输出长度预测的代理模型

问题建模。我们把每个查询的输出 token 长度预测建模为回归问题 (方案 #1), 或多分类问题 (方案 #2, 与 S^3 [14] 类似), 见图 2(a)。回归任务预测绝对输出长度; 分类任务预测基于百分位值划分的类别。例如, 二分类可以使用 $[0, p50]$ 和 $[p50, \max]$ 两类, 其中 p50 是历史 LLM 服务记录的输出长度中位数。

另一种建模是成对预测器, 见图 2(b): 输入两个 LLM 查询 Q_1, Q_2 ; Q_1 输出比 Q_2 长时预测 1, 否则预测 0。利用该预测器, SJF 可以比较新到请求与队列中已有请求, 将其插入按输出长度排序的队列。然而, 这种方法每次插入可能需要 $O(\log N)$ 次代理模型推理, 其中 N 是最大队列长度, 而 SSJF 的单查询预测只需 $O(1)$ 次。此外, 成对预测 SJF 相对于 FCFS 的 JCT 改善只有 $\sim 3\%$, SSJF 的改善幅度为其 $> 10\times$ 。因此, 我们没有继续采用成对预测方法。

预测器架构。如图 2(a) 所示, 我们使用 BERT-base 作为代理模型, 并在其后追加带有 softmax 层的两层全连接神经网络, 形成输出 token 长度预测器。具体地, 取 BERT 输出中第一个 token, 即 CLS, 在最后一层的隐藏状态。按照 BERT 论文 [7] 的建议, 将 CLS 的最终隐藏状态作为分类任务的整体序列表示。

代理模型微调。从 [7] 提供的预训练 BERT-base 开始, 我们在 LMSYS-Chat-1M 训练数据 [30] 上微调 BERT 及新增两层预测网络的权重。大规模无标签文本预训练使代理模型具有基础语言理解能力, 微调使它学习任务相关知识, 即根据查询预测生成模型的输出长度。微调分两阶段: 第一阶段同时调整 BERT-base 与预测器的参数; 第二阶段固定 BERT-base, 只更新预测器参数。两阶段方案较好地平衡了预测准确性与训练效率。

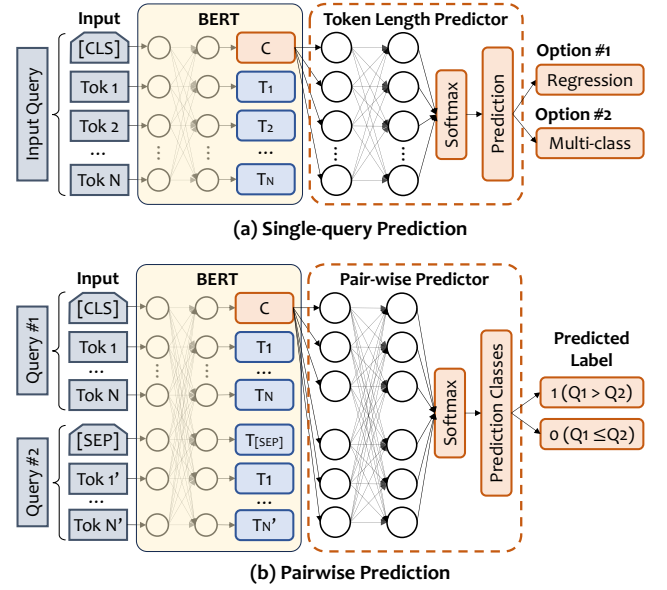


图 2: 输出长度预测器架构。(a) 为单查询预测, (b) 为成对预测。图内英文: Input、Input Query 为输入和输入查询, Tok 为 token, [CLS]、[SEP] 分别为分类与分隔标记; Token Length Predictor 为 token 长度预测器, Softmax 为归一化函数, Prediction 为预测, Regression 为回归, Multi-class 为多分类, Option 为方案; Pair-wise Predictor 为成对预测器, Prediction Classes 为预测类别, Predicted Label 为预测标签, Query 1、Query 2 为两个查询。标签 1 表示 Q_1 的输出比 Q_2 长, 0 表示不长于 Q_2 。

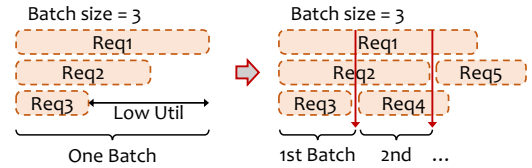


图 3: 组批与连续组批的比较。图内英文: Req 为请求, Batch size 为批大小, Low Util 为低利用率, One Batch 为一个批次, 1st Batch 为第一批次, 2nd 为第二批次。左侧整个批次等待长请求, 右侧以新请求填补已完成请求的位置。

对于回归建模 (方案 #1), 可以采用 L1 损失或 MSE 损失; 对于多分类建模 (方案 #2), 可以采用标准交叉熵损失, 也可以采用有序分类训练 [25]。

例如, 在五分类中, 把类别转为 5 个整数 $\{0, 1, 2, 3, 4\}$, 让预测器输出一个实数。随后用 L1 或 MSE 损失, 最小化实数预测与整数类别标签的距离; 推理时, 将预测值舍入到 $\{0, \dots, 4\}$ 中最近的整数。§4 表明, 使用 L1 损失的回归 (REG-L1) 获得了最佳预测准确率与调度性能。

支持多轮 LLM 对话。除一般 LLM 推理外, SSJF 也支持多轮交互。此前各轮的用户查询与 LLM 回答, 通常作为当前查询的上下文。然而, BERT-base 代理模型的输入上限为 512 个 token, 可能远小于 LLM 的上下文窗口。为支持多轮对话, 我们采用拼接与截断: 将此前各轮用户提示 P_1, P_2, P_3 等、模型响应 R_1, R_2, R_3 等, 以及当前用户提示拼接为增强上下文的查询,

只保留最后 512 个 token，丢弃较早部分。这样，代理模型在输入长度限制内仍能处理多轮对话。

3.2 调度与组批

预测每个到达请求的输出长度后，SSJF 让预测较短的请求先运行。它支持无组批、动态组批、连续组批三种模式。无组批时，按 SSJF 确定的顺序逐个处理请求。动态组批时，调度器等待 batch_wait_timeout 到期，或直到每批填满 max_batch_size 个请求；请求顺序由 SSJF 决定。连续组批最早由 Orca [28] 提出，按迭代组批；提前完成的请求由等待队列中的下一请求替换，队列按 SSJF 排序。动态组批与连续组批的差别见图 3。

4 评估

4.1 实验设置

模型。SSJF 是一个通用机器学习模型服务框架，因此评估包括传统非 Transformer 模型、Transformer，以及基于 Transformer 的生成式模型。对每个模型家族，选取若干常用规模与变体，以模拟不同微调版本。表 1 列出模型大小与测试平台 GPU 上的推理时延。

工作负载。使用 Microsoft Azure 函数轨迹 [29] 驱动推理工作负载。语言模型输入来自 LMSYS-Chat-1M [30]，包含与 25 个先进 LLM 的 100 万次真实对话。

测试平台。我们将 SSJF 部署于 IBM Cloud 的 gx2-16x128x2v100 实例，含两张 16 GB NVIDIA Tesla V100。每张 GPU 的流多处理器 (SM) 最高频率为 1380 MHz，最低为 200 MHz。

4.2 调度性能

我们通过作业完成时间 (JCT) 和服务吞吐量评估调度，检验预测准确性是否足够。如 §3.1 所述，预测器使用 L1 损失的回归，因为在下节消融实验 §4.3 中准确率更高。基线包括：1) FCFS，即 Orca [28]、vLLM [15] 等先进服务框架的默认调度器；2) 使用实际输出长度的 SJF，即理想预测器。

请求服务的 JCT。我们在无组批、动态组批、连续组批三种设置下评估 SSJF 和基线；后两者的最大批大小均为 4。图 4 和图 5 展示不同到达速率、突发程度下的平均 LLM 服务 JCT。改变请求速率时，相对于 FCFS，SSJF 在三种设置下分别降低 JCT 34.5%、39.6%、33.2%；理想 SJF 分别降低 43.7%、58.2%、43.0%。改变突发程度时，SSJF 相对于 FCFS 分别降低 30.5%、39.0%、35.0%，理想 SJF 分别降低 37.6%、52.9%、41.5%。

请求服务的吞吐量。相同实验设置的吞吐量结果见图 6 和图 7。改变请求速率时，SSJF 在无组批、动态组批、连续组批下，将吞

表 1: 实验使用的模型。时延在单张 GPU 上，对序列长度为 512 的查询测量。

模型	参数量	大小	时延
OPT-1.3b	1.3 B	5.0 GB	1243 ms
OPT-2.7b	2.7 B	10.4 GB	2351 ms
GPT2-large	774 M	3.3 GB	832 ms
GPT2-xl	1.5 B	6.4 GB	1602 ms
CodeGen-350m	350 M	1.3 GB	357 ms
CodeGen-2b	2.0 B	8.0 GB	2507 ms
Bloom-1b1	1.1 B	4.0 GB	523 ms
Bloom-3b	3.0 B	11.0 GB	1293 ms
Switch-base-16	920 M	2.4 GB	348 ms
Switch-base-32	1.8 B	4.8 GB	402 ms

表 2: 输出 token 长度预测器的评估结果。SSJF 采用 L1 回归，即 REG (L1)，因为其准确率与调度改善更好。表中 Bin. CLS 为二分类，Ord. CLS 为有序分类，CLS 为标准分类，REG 为回归，MSE 为均方误差。

指标	二分类	有序分类 (MSE)	有序分类 (L1)	分类	回归 (MSE)	回归 (L1)
准确率 ↑	N/A	0.477	0.580	0.592	0.601	0.615
F1 得分 ↑	N/A	0.450	0.576	0.590	0.602	0.615
JCT 降幅 ↑	30.8%	38.0%	35.6%	35.8%	38.6%	39.1%
吞吐量 ↑	1.59×	2.15×	1.99×	1.98×	2.19×	2.21×

吐量分别提高至 3.6×、3.0×、2.8×；理想 SJF 分别为 4.7×、4.1×、3.2×。改变突发程度时，SSJF 分别为 2.6×、2.6×、2.2×，理想 SJF 分别为 3.4×、3.8×、2.7×。

不同批大小。我们增大批大小并重复实验，结果见图 8。连续组批始终比动态组批具有更低 JCT、更高吞吐量，与 [28] 的结果一致。改变批大小时，SSJF 的调度优于 FCFS，但随批次增大，它带来的 JCT 和吞吐量改善减少，因为批次越大，重新排列队列造成的差别越小。

不同对话轮次。我们评估 SSJF 对多轮 LLM 对话的支持效果，见图 9。相对于 FCFS，不同轮次的 JCT 改善从第 #5 轮的 33.1% 到第 #1 轮的 38.9% 不等；吞吐量倍率从第 #5 轮的 1.67× 到第 #1 轮的 2.22× 不等。

预测器开销。我们比较预测器，即代理模型推理的时延开销与 LLM 请求执行时间。平均模型执行时间为 9.8 s，大于平均预测时延 7.6 ms；原文称后者占总时延的 0.02%。最大预测时延为 20.2 ms，仍小于最短模型执行时间 120 ms。因此，预测器对端到端 LLM 服务时延的开销很小，支持了 SSJF 使用轻量代理模型的设计。¹

4.3 LLM 输出长度预测

如 §3.1 所述，消融研究表明使用 L1 损失的回归准确率更高，因此最终采用它。本节评估预测器准确性，并对预测方法与训练方法进行消融。

所有模型上的预测准确率。我们在 LMSYS-Chat-1M [30] 上评估长度预测器。所有模型的平均准确率为 0.615；这里将输出转换为五分类评价，随机猜测准确率为 0.2。如图 9 所示，不同轮次的准确率从第 #2 轮的 0.60 到第 #1 轮的 0.619 不等；F1 从第 #2 轮的 0.599 到第 #1 轮的 0.618 不等。

消融研究。表 2 比较多种替代方法，包括二分类、采用 MSE/L1 损失的有序多分类、采用交叉熵的标准分类，以及采用 MSE 的回归。SSJF 所用的 L1 回归，在预测准确率、F1、平均 JCT 改善和吞吐量改善上均最好。

数据处理说明。LMSYS-Chat-1M [30] 中因不明原因存在空回答，以及随机截断的回答；截断只发生于部分长度大于 512 的回答。因此，我们按响应长度 (1, 512) 筛选数据。筛选前，采用 MSE 的多分类准确率为 0.42，优于其他所有预测方法。因此，预测方法的选择，包括回归还是分类、类别数量，可以依赖数据集或模型。

5 相关工作

LLM 输出长度预测。S³ [14] 基于 DistilBERT (由 BERT-base 蒸馏得到的 Transformer) 建立多分类器，把查询的输出 token 长度分到 10 个等宽桶。TetriInfer [12] 类似地使用分类模型，即 1.25 亿参数 OPT，预测长度。然而，S³ 不支持多轮对话，训练

¹译注：7.6 ms 除以 9.8 s 约为 0.078%，并非 0.02%。若原文的“总时延”还包括较长排队等开销，则分母可能不同，但本段没有说明这一计算口径。

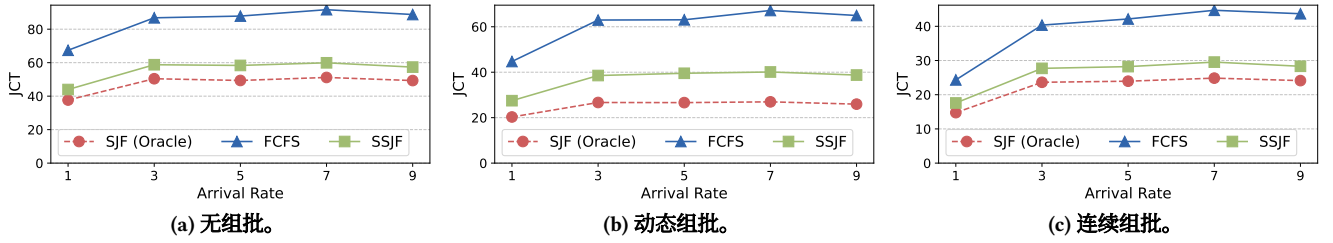


图 4: 不同请求速率下的作业完成时间 (JCT)。图内英文: Arrival Rate 为到达速率, JCT 为作业完成时间; FCFS 为先到先服务, SSJF 为本文调度器, SJF (Oracle) 为使用实际长度的理想最短作业优先。

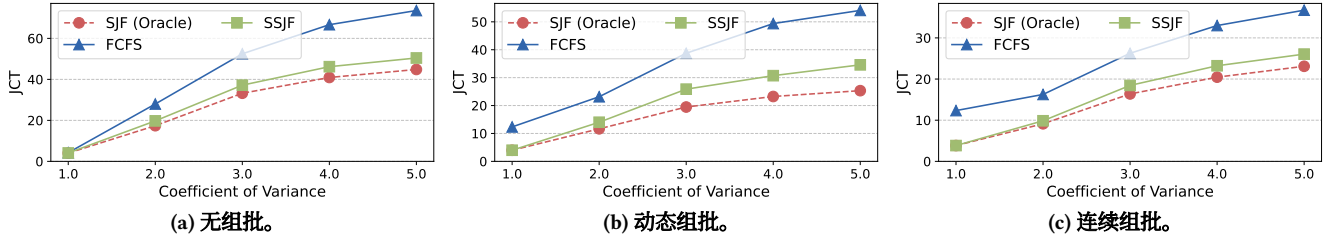


图 5: 不同突发程度下的作业完成时间 (JCT)。图内英文: Coefficient of Variance 为变异系数, 用于表示到达的突发程度; JCT 为作业完成时间, 图例为 FCFS、SSJF 与实际长度 SJF (Oracle)。

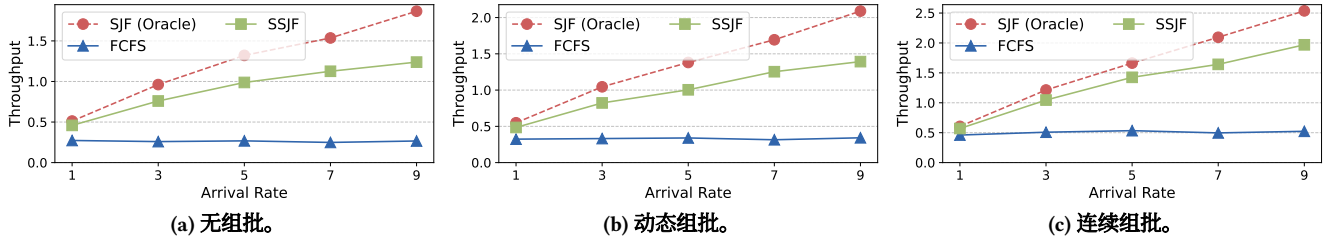


图 6: 不同请求速率下的吞吐量。图内英文: Arrival Rate 为到达速率, Throughput 为吞吐量; 图例为 FCFS、SSJF 与实际长度 SJF (Oracle)。

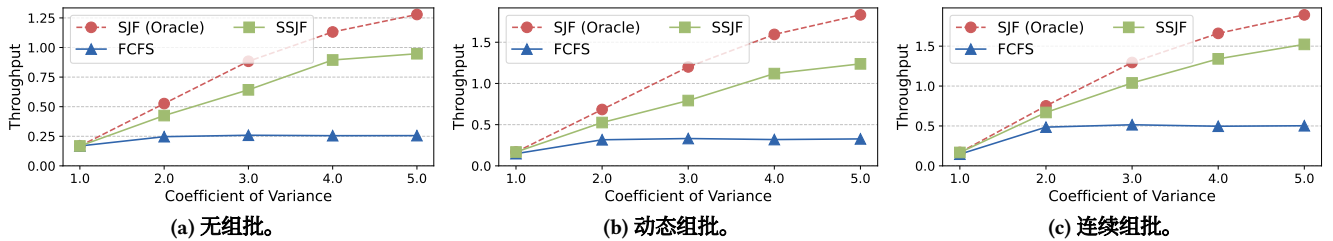


图 7: 不同突发程度下的吞吐量。图内英文: Coefficient of Variance 为表示突发程度的变异系数, Throughput 为吞吐量; 图例为 FCFS、SSJF 与实际长度 SJF (Oracle)。

只采用一个 52K 问答数据集; 原文称其约为本文使用 LMSYS-Chat-1M 数据的 ~2%。我们在更大数据集上的实验表明, 相对于多分类, SSJF 的回归进一步改善调度 JCT 与吞吐量。Zheng 等人 [31] 则提示 LLM 自身预测长度, 但修改同一用户提示可能引入偏差, 且相对于代理模型方法开销更高。²

基于预测的 SJF。在机器学习服务之外, 大数据计算集群 [13, 27] 与高性能计算集群 [19, 32] 已有基于预测的 SJF 调度器。例如, SLearn [13] 根据相似作业的历史估计运行时间, 在 Hadoop 上进行 SJF 调度; PREP [32] 使用支持向量回归, 根据作业运行状

²译注: 若直接以数据集名称中的 100 万条作为分母, 52K 对应 5.2%, 不是 2%。原文没有在此交代用于这一比例的实际数据量, 故保留其原值。

态、资源分配、用户或组 ID 等特征估计 HPC 作业时间。相对于 HPC、Hadoop 类集群, 本文代理模型序列长度预测针对自回归模型独有的执行时间不确定性, 支持高效交互式 LLM 推理。

6 讨论与未来工作

饥饿处理与抢占。为了避免 SJF 中的饥饿, 可以采用老化 [23], 提升等待较长的作业。抢占也可以配合最小松弛时间优先策略, 修正此前次优的决定。然而, SSJF 没有采用抢占, 因为上下文切换 [2] 和内存管理 [15] 会增加复杂性。

使用语义缓存。为提高长度预测准确性, 可以使用简单的语义缓存, 例如 GPTCache [3], 缓存热门输入的真实输出长度。当

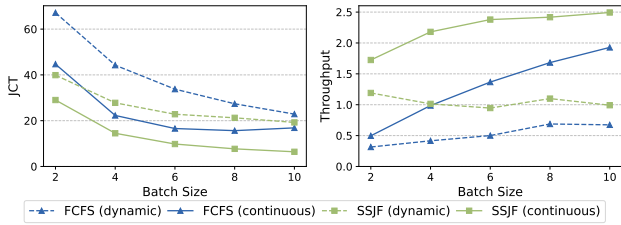


图 8: 不同批大小下 SSJF 的收益。图内英文: Batch Size 为批大小, JCT 为作业完成时间, Throughput 为吞吐量; dynamic、continuous 分别为动态与连续组批, 图例比较两种组批下的 FCFS 和 SSJF。

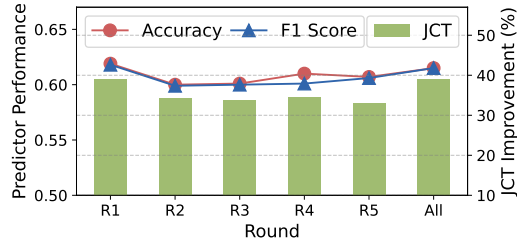


图 9: SSJF 在不同对话轮次上的评估。图内英文: Round 为轮次, All 为全部轮次, Predictor Performance 为预测器表现, Accuracy 为准确率, F1 Score 为 F1 得分, JCT Improvement (%) 为作业完成时间改善百分比; 柱形使用右侧 JCT 改善轴。

缓存中存在语义相近的查询而触发命中时, 调度器直接取缓存长度, 而不经预测器估计。

代理模型的其他使用机会。轻量代理模型预测请求输出长度后, 还有多种潜在用途。第一, 模型服务系统可以动态分配合适的输出内存, 减少过量分配引起的低利用率; 预测还可优化生成输出的缓冲区大小, 减少重新分配或不必要内存使用。第二, 可以指导输出缓存策略: 更积极缓存短输出, 对长输出选择性缓存或更早逐出, 以节约缓存空间。第三, 将预测长输出的请求路由到更强或专用服务器、集群, 让短请求由资源较少的实例处理。第四, 可结合 KV 缓存访问模式, 辅助将 GPU 内存卸载到 CPU 内存或磁盘。但必须仔细处理预测错误。

与推测解码技术集成。推测解码或前瞻解码, 使用小型近似模型 [16, 18]、多个解码头 [4], 或 n-gram 生成 [8] 来加快 LLM token 生成。由于候选生成和验证会增加计算开销, 本文没有考虑推测式 LLM 推理。

与 vLLM 集成。在 vLLM [24] 中, 从 v0.6.2 开始, 可以利用优先级调度器, 将预测序列长度作为每个到达请求的优先级标签。这使请求优先级能根据预计计算需求, 即序列长度, 动态调整。

7 结论

本文提出面向交互式 LLM 应用的高效模型服务调度器 SSJF, 核心是利用基于代理模型的序列长度预测。在真实 LLM 数据集与生产工作负载轨迹上的评估表明, SSJF 在无组批、动态组批、连续组批下, 将 JCT 改善 30.5–39.6%, 吞吐量提高至 2.2–3.6 $\times$ 。SSJF 的开源实现位于 <https://github.com/JamesQiuHaoran/LLM-serving-with-proxy-models>。

致谢

感谢匿名审稿人提供宝贵反馈。本工作获得美国国家科学基金会 (NSF) CCF 20-29049 项目和 IBM-ILLINOIS Discovery

Accelerator Institute (IIDAI) 的支持。文中的观点、发现、结论和建议属于作者, 不一定代表 NSF 或 IBM 的意见。

参考文献

- [1] Triton inference server. <https://developer.nvidia.com/triton-inference-server>.
- [2] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. PipeSwitch: Fast pipelined context switching for deep learning applications. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2020)*, pages 499–514, 2020.
- [3] Fu Bang and Feng Di. GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, pages 212–218, 2023.
- [4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, and Tri Dao. Medusa: Simple framework for accelerating LLM generation with multiple decoding heads, 2023. <https://github.com/FasterDecoding/Medusa>.
- [5] Daniel Crankshaw, Gur-Eyal Sela, Xiangxi Mo, Corey Zumar, Ion Stoica, Joseph Gonzalez, and Alexey Tumanov. InferLine: Latency-aware provisioning and scaling for prediction serving pipelines. In *Proceedings of the 11th ACM Symposium on Cloud Computing*, page 477–491, New York, NY, USA, 2020. Association for Computing Machinery.
- [6] Daniel Crankshaw, Xin Wang, Guilio Zhou, Michael J Franklin, Joseph E Gonzalez, and Ion Stoica. Clipper: A low-latency online prediction serving system. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2017)*, pages 613–627, 2017.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional Transformers for language understanding. In *Proceedings of the 2019 North American Chapter of the Association for Computational Linguistics (ACL 2019)*, 2019.
- [8] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding, 2023. <https://lmsys.org/blog/2023-11-21-lookahead-decoding/>.
- [9] GitHub. GitHub Copilot. <https://github.com/features/copilot>, 2024. Accessed: 2024-03-18.
- [10] Google. An important next step on our AI journey. <https://blog.google/technology/ai/bard-google-ai-search-updates/>, 2023. Accessed: 2024-03-18.
- [11] Arpan Gujartati, Reza Karimi, Safya Alzayat, Wei Hao, Antoine Kaufmann, Ymir Vigfusson, and Jonathan Mace. Serving DNNs like Clockwork: Performance predictability from the bottom up. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462, 2020.
- [12] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Jiang Xu, Shuang Chen, Hao Feng, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Inference without interference: Disaggregate LLM inference for mixed downstream workloads, 2024.
- [13] Akshay Jajoo, Y. Charlie Hu, Xiaojun Lin, and Nan Deng. A case for task sampling based learning for cluster job scheduling. In *Proceedings of the 19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 2022)*, pages 19–33, Renton, WA, 2022. USENIX Association.
- [14] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. S³: Increasing GPU utilization during generative inference for higher throughput. *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 36, 2023.
- [15] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*, pages 611–626, 2023.
- [16] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)*, pages 19274–19286. PMLR, 2023.
- [17] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E Gonzalez, et al. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2023)*, pages 663–679, 2023.
- [18] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Rae Ying Yee Wong, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating generative LLM serving with speculative inference and token tree verification. *arXiv preprint arXiv:2305.09781*, 2023. <https://arxiv.org/abs/2305.09781>.
- [19] Mina Naghshnejad and Mukesh Singhal. A hybrid scheduling platform: a runtime prediction reliability aware scheduling platform to improve HPC scheduling performance. *The Journal of Supercomputing*, 76:122–149, 2020.
- [20] Christopher Olston, Fangwei Li, Jeremiah Harmsen, Jordan Soyke, Kiril Gorovoy, Li Lao, Noah Fiedel, Sukriti Ramesh, and Vinu Rajashekhar. TensorFlow-Serving: Flexible, high-performance ML serving. In *Workshop on ML Systems 2017*, 2017.

- [21] OpenAI. Introducing ChatGPT. <https://openai.com/blog/chatgpt>, 2022. Accessed: 2024-03-18.
- [22] George Prekas, Marios Kogias, and Edouard Bugnion. Zygos: Achieving low tail latency for microsecond-scale networked tasks. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP 2017)*, pages 325–341, 2017.
- [23] Yonatan Shadmi. Fluid limits for shortest job first with aging. *Queueing Systems*, 101(1-2):93–112, 2022.
- [24] vLLM: A high-throughput and memory-efficient inference and serving engine for LLMs. <https://github.com/vllm-project/vllm>, 2024.
- [25] Christopher Winship and Robert D Mare. Regression models with ordinal variables. *American Sociological Review*, pages 512–525, 1984.
- [26] Bingyang Wu, Yinmin Zhong, Zili Zhang, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. *arXiv preprint arXiv:2305.05920*, 2023.
- [27] Yi Yao, Jianzhe Tai, Bo Sheng, and Ningfang Mi. LsPS: A job size-based scheduler for efficient task assignments in Hadoop. *IEEE Transactions on Cloud Computing*, 3(4):411–424, 2015.
- [28] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*, pages 521–538, 2022.
- [29] Yanqi Zhang, Ñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh El-nikety, Christina Delimitrou, and Ricardo Bianchini. Faster and cheaper serverless computing on harvested resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP 2021)*, pages 724–739, 2021.
- [30] Lianmin Zheng, Wei-Lin Chiang, et al. LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. *arXiv preprint arXiv:2309.11998*, 2023.
- [31] Zangwei Zheng, Xiaozhe Ren, Fuzhao Xue, Yang Luo, Xin Jiang, and Yang You. Response length perception and sequence scheduling: An LLM-empowered LLM inference pipeline. *Advances in Neural Information Processing Systems (NeurIPS 2023)*, 36, 2023.
- [32] Longfang Zhou, Xiaorong Zhang, Wenxiang Yang, Yongguo Han, Fang Wang, Yadong Wu, and Jie Yu. PREP: Predicting job runtime with job running path on supercomputers. In *Proceedings of the 50th International Conference on Parallel Processing (ICPP 2021)*, New York, NY, USA, 2021. Association for Computing Machinery.